React

Ashley Darling
Spelman College
ashleydarling@spelman.edu

Wynter Stokes
Spelman College
wynterstokes@spelman.edu

Imani Candler
Spelman College
imanicandler@spelman.edu

ABSTRACT

This paper explores the development of an interactive educational game, **Are You Smarter Than a Fifth Grader?**, using two distinct programming paradigms: **Scheme** (a functional programming language) and **React** (a JavaScript-based front-end framework). We provide a comparative analysis of the languages, evaluate the code structure in both implementations, and present a detailed program design and decomposition. The paper highlights the strengths and limitations of each language, providing insights into their suitability for building interactive educational applications.

1. INTRODUCTION

Interactive learning tools have become popular in educational settings as they engage users while testing their knowledge. We developed a prototype for the game **Are You Smarter Than a Fifth Grader?** in two different programming languages: **Scheme** and **React**. This project aims to compare the two implementations, analyze their differences, and evaluate their effectiveness in creating interactive experiences.

1.1 Motivation

By comparing a traditional functional programming language (Scheme) with a modern front-end framework (React), we aim to showcase how different paradigms can be utilized in developing educational games, exploring the trade-offs and benefits of each approach.

2. COMPARISON OF LANGUAGES

2.1 Scheme: Functional Programming Paradigm

- **Type:** Functional, dynamically typed.
- **Key Features:** Recursion, first-class functions, immutability.
- **Strengths:** Concise code, strong emphasis on recursion & abstraction.
- **Limitations:** Limited built-in support for user interfaces & web development.

2.2 React/JavaScript: Component-Based Front-End Framework

- **Type:** Multi-paradigm (supports object-oriented & functional styles).
- **Key Features:** Component-based architecture, virtual DOM, state management.
- **Strengths:** Responsive UI, reusable components, large ecosystem.
- **Limitations:** Requires a setup environment, reliance on JavaScript.

2.3 Summary of Comparison

- **Ease of Development:** React offers more tools & libraries for building user interfaces, while Scheme focuses on concise, functional logic.
- **Interactivity:** React excels in building interactive & responsive UIs. Scheme is better suited for logic-heavy, algorithmic tasks.
- **Learning Curve:** Scheme is often used in educational contexts for its emphasis on recursion. React has a steeper learning curve but provides more extensive features for building dynamic web applications.

React

Ashley Darling
Spelman College
ashleydarling@spelman.edu

Wynter Stokes
Spelman College
wynterstokes@spelman.edu

Imani Candler
Spelman College
imanicandler@spelman.edu

3. COMPARISON OF CODE

3.1 Code Example in Scheme

```
(define (validate-answer user-answer correct-answer)
  (if (equal? user-answer correct-answer)
      'correct
      'incorrect))

(display (validate-answer "Paris" "Paris")) ; Output: correct
```

3.2 Code Example in React/JavaScript

```
import React, { useState } from 'react';
function AnswerValidator({ userAnswer, correctAnswer }) {
  const [result, setResult] = useState('');
  const validateAnswer = () => {
    if (userAnswer.toLowerCase() === correctAnswer.toLowerCase()) {
      setResult('Correct!');
    } else {
      setResult('Incorrect!');
    }
  };

  return (
    <div>
      <button onClick={validateAnswer}>Submit Answer</button>
      <p>{result}</p>
    </div>
  );
}

export default AnswerValidator;
```

3.3 Analysis

- **Conciseness:** Scheme provides a more compact representation for simple logic but lacks user interface capabilities.
- **Interactivity:** React's code focuses on user interaction & dynamic feedback, leveraging the framework's strengths.

4. OVERVIEW OF REACT/JAVASCRIPT

React is a JavaScript library for building interactive user interfaces. It uses a **component-based architecture**, allowing developers to create reusable components with their own state and behavior. React uses the **virtual Document Object Model (DOM)** to efficiently update the user interface in response to state changes, making it ideal for dynamic, real-time applications.

4.1 Key Features

- **JSX Syntax:** Combines JavaScript & HTML-like syntax for building UI components.
- **State Management:** Uses hooks like *useState* & *useEffect* for managing state & side effects.
- **Component Reusablility:** Promotes reusable, isolated components that manage their own logic.

4.2 Benefits

React's ecosystem includes numerous libraries (like Rodux for state management), making it suitable for complex applications. It offers excellent support for building responsive, interactive web interfaces.

5. PROGRAM DESIGN & DECOMPOSITION

The program is designed around a modular structure with clearly defined components.

5.1 Scheme Implementation

- **Main Game Loop:** Manages the flow of the game & prompts the user with questions.
- **Question Function:** Displays a question & checks the user's answer using validate-answer.
- **Score Tracker:** Keeps tracks of the user's correct & incorrect answers, displays the final score.

5.2 React Implementation

- **App Component:** Serves as the main entry point, managing the game state.

React

Ashley Darling
Spelman College
ashleydarling@spelman.edu

Wynter Stokes
Spelman College
wynterstokes@spelman.edu

Imani Candler
Spelman College
imanicandler@spelman.edu

- **QuestionDisplay Component:** Renders the current question & answer options.
- **AnswerValidator Component:** Checks the user's input and updates the result.
- **ScoreDisplay Component:** Shows the current score.
- **GameEndScreen Component:** Displays the final score & congratulatory message.

5.3 Comparsion of Design

- Scheme focuses on recursion & functional decomposition, emphasizing logic & flow control.
- React's design relies on components & hooks, emphasizing state management & user interaction.

6. EXPLANATION OF THE DEMO PROGRAMMING PROBLEM

The demo programming problem is a simplified version of **Are You Smarter Than a Fifth Grader?**, where users answer a series of questions from different categories (e.g., mathematics, history). The game validates answers & provides feedback, accumulating a score based on correct answers.

6.1 Goals

- Implement core game logic in both Scheme & React.
- Provide interactive feedback in the React version, enhancing user experience through dynamic UI updates.
- Showcase functional programming principles in Scheme & component-based architecture in React.

6.2 User Flow in React

1. **Start Game:** Users begin the quiz by clicking the "Start" button.
2. **Question Display:** The app shows a question with multiple-choice options using radio buttons.
3. **Answer Submission:** Users select an answer & click "Submit," triggering validation.
4. **Feedback Display:** The app shows "Correct!" or "Incorrect!" based on the user's input.
5. **Game End:** The app displays the final score with an option to restart.

7. CONCLUSION

The comparison of Scheme & React highlights the diverse approaches offered by different programming paradigms in developing interactive educational applications. While Scheme provides a concise, logic-focused approach, React excels in creating dynamic, user-friendly interfaces. This project demonstrates the potential of combining functional programming concepts with modern web development frameworks to build engaging & educational tools.

8. FUTURE WORK

Future enhancements could include:

- Adding a backend for user authentication & a leaderboard.
- Expanding the question set & integrating an application programming interface (API) for dynamic questions.

React

Ashley Darling
Spelman College
ashleydarling@spelman.edu

Wynter Stokes
Spelman College
wynterstokes@spelman.edu

Imani Candler
Spelman College
imanicandler@spelman.edu

- Creating a more sophisticated UI with animations & sound effects in the React version.

9. ACKNOWLEDGEMENTS

Special thanks to the entire React team for their contributions.

10. REFERENCES

[1] *What is React?*. W3Schools. Accessed October 2024. https://www.w3schools.com/whatis/whatis_react.asp.
[2] React. Accessed October 2024. https://react.dev/.
[3] *React Tutorial*. W3Schools. Accessed October 2024. https://www.w3schools.com/react/.
[4] Documentation on Scheme & functional programming.
[5] React.js documentation for front-end development.